

NEXO-UD

Plataforma de Apoyo Académico

Universidad Distrital Francisco José de Caldas

Manual del Desarrollador

Fuente única de verdad técnica para la inducción de nuevos

colaboradores.

Toda la información fue extraída directamente del código fuente,
configuraciones y migraciones del repositorio.

Mayo de 2026

Índice

1. Visión General de la Arquitectura	3
1.1. Patrón Arquitectónico	3
1.2. Stack Tecnológico	4
1.2.1. Backend	4
1.2.2. Frontend	4
2. Configuración y Entorno Local	4
2.1. Requisitos Previos	5
2.2. Estructura de Carpetas del Repositorio	5
2.3. Configuración de la Base de Datos	5
2.4. Variables de Entorno	6
2.4.1. Backend — <code>nexo-backend/.env</code>	6
2.4.2. Frontend — <code>nexo-frontend/.env</code>	6
2.5. Pasos para Levantar el Entorno Local	6
3. Topología del Dominio y Modelo de Datos	8
3.1. Entidades Principales	8
3.1.1. <code>users</code> — Usuario	8
3.1.2. <code>user_roles</code> — Roles de Usuario	8
3.1.3. <code>verification_codes</code> — Códigos de Verificación	9
3.1.4. Oferta Académica — Jerarquía	9
3.1.5. Horarios Guardados	9
3.1.6. Campus	9
3.1.7. Tipos Enum de Dominio	10
3.2. Diagrama de Relaciones Clave	10
4. Diseño de la Capa de Servicios y Lógica de Negocio	10
4.1. Inventario de Servicios	11
4.2. Lógica de Negocio Destacada	11
4.2.1. Registro de Estudiante (<code>AuthServiceImpl.register</code>)	11
4.2.2. Sistema de Códigos de Verificación	12
4.2.3. Exportación de Horario (<code>ScheduleExportServiceImpl</code>)	12
4.3. Gestión de Transacciones	12
4.4. Excepciones Personalizadas	12
5. Contratos de Integración (API Layer)	13
5.1. Autenticación JWT	13
5.2. Endpoints Públicos (sin autenticación)	13
5.3. Resumen de Controllers y Rutas	14
5.3.1. <code>AuthController</code> — <code>/api/v1/auth</code>	14
5.3.2. <code>UserController</code> — <code>/api/v1/users</code>	14
5.3.3. <code>SchedulePlannerController</code> — <code>/api/v1/schedules</code>	15
5.3.4. <code>CampusController</code> — <code>/api/v1/campus</code>	15
5.3.5. <code>ReportController</code> — <code>/api/v1/reports</code>	16
5.3.6. <code>SemesterController</code>	16

5.4.	Paginación	16
5.5.	DTOs — Ejemplos de Request/Response	16
5.5.1.	Login	16
5.5.2.	Registro	17
5.5.3.	Validar conflictos de horario	17
5.6.	Capa de API en el Frontend	17
6.	Estrategia de Pruebas	18
6.1.	Stack de Pruebas	18
6.2.	Convenciones de Nomenclatura de Tests	19
6.3.	Comandos para Ejecutar Tests	19
6.4.	Tests con Spring Security (@WebMvcTest)	19
7.	Estándares y Flujo de Contribución	20
7.1.	Convenciones de Código	20
7.1.1.	Backend (Java)	20
7.1.2.	Estructura de Cada Módulo	20
7.2.	Estructura de Ramas	21
7.3.	Formato de Commits (Conventional Commits)	21
7.4.	Checklist Antes de Hacer PR	21
7.5.	Agregar una Nueva Migración Flyway	21
7.6.	Agregar un Nuevo Módulo de Dominio	22
8.	Gestión de Estado del Frontend	22
8.1.	Estrategia General	22
8.2.	AuthContext — Flujo de Restauración de Sesión	22
8.3.	Limitaciones Conocidas y Mejoras Futuras	23
9.	Infraestructura y Despliegue	23
9.1.	Estado Actual de la Infraestructura	24
9.2.	Dockerfile del Backend	24
9.3.	Variables de Entorno Requeridas en Producción	24
9.4.	CI/CD — Estado y Recomendación	25
10.	Observabilidad y Logs	25
10.1.	Configurar Niveles de Log	26
10.2.	Endpoints de Actuator	26
10.3.	Monitoreo Futuro Recomendado	26
11.	Troubleshooting — Errores Comunes	26
11.1.	Backend	26
11.2.	Frontend	28

1. Visión General de la Arquitectura

1.1 Patrón Arquitectónico

NEXO-UD sigue una arquitectura **monolítica por módulos** (*Modular Monolith*) con separación estricta por dominios de negocio. Cada dominio tiene su propio paquete con capas internas bien definidas:

```
com.kumorai.nexo/  
    auth/          -> Autenticacion, registro, verificacion  
    por codigo  
    user/          -> Gestion de usuarios, roles, progreso  
    academico  
    academic/      -> Oferta academica, planes de estudio,  
    semestres  
    schedule/      -> Armador y exportador de horarios  
    campus/        -> Sedes, aulas, fotos, rutas de  
    transporte  
    content/        -> Avisos, calendario academico, bienestar  
    report/         -> Reportes y retroalimentacion de  
    estudiantes  
    admin/          -> Operaciones administrativas  
    transversales  
    shared/         -> Configuracion, seguridad, utilidades,  
    excepciones
```

Cada módulo sigue el patrón MVC en capas:

Controller → Service (interface + impl) → Repository (Spring Data JPA)
→ Entity (JPA)

El **frontend** es una SPA (*Single Page Application*) con React que consume la API REST del backend a través de un cliente HTTP centralizado.

1.2 Stack Tecnológico

1.2.1 Backend

Componente	Tecnología	Versión
Lenguaje	Java	17
Framework principal	Spring Boot	3.5.14
Seguridad	Spring Security	6.x
Persistencia (ORM)	Spring Data JPA / Hibernate	6.x
Base de datos	PostgreSQL	15+ recomendado
Migraciones	Flyway	incluido en Boot 3.5
Autenticación	JWT (jjwt-api)	0.12.6
Generación de PDF	OpenPDF (librepdf)	1.3.30
Email	Spring Boot Mail (SMTP Gmail)	incluido
Build	Maven	3.9+
Lombok	Lombok	incluido en Boot 3.5
Validación	Jakarta Validation	incluido

Cuadro 1: Stack del backend

1.2.2 Frontend

Componente	Tecnología	Versión
Lenguaje	TypeScript	6.0.3
Framework UI	React	18.3.1
Build tool	Vite	6.3.5
Estilos	Tailwind CSS	4.1.12
Componentes base	Radix UI	varios
Enrutamiento	React Router	7.x
Formularios	React Hook Form	—
Mapas	Leaflet + react-leaflet	—
Gráficas	Recharts	—
Temas	next-themes	—

Cuadro 2: Stack del frontend

2. Configuración y Entorno Local

2.1 Requisitos Previos

Herramienta	Versión mín.	Propósito
JDK	17	Compilar y ejecutar el backend
Maven	3.9	Gestionar dependencias y ciclo de vida del backend
Node.js	20 LTS	Compilar y ejecutar el frontend
npm	10+	Gestor de paquetes del frontend
PostgreSQL	15	Base de datos relacional
Git	2.40+	Control de versiones

Cuadro 3: Requisitos previos de desarrollo

Recomendado: Usar [SDKMAN](#) para gestionar versiones de Java y Maven en Linux/macOS.

2.2 Estructura de Carpetas del Repositorio

```

NEXO-UD/
├── nexo-backend/      -> Spring Boot API REST
│   ├── src/
│   │   ├── main/java/com/kumorai/nexo/
│   │   ├── main/resources/
│   │   │   ├── application.properties
│   │   │   └── db/migration/      -> Migraciones
│   └── Flyway (V1 a V13)
│       └── test/java/com/kumorai/nexo/
│           ├── pom.xml
│           └── nexo-frontend/      -> React SPA
│               ├── src/
│               │   ├── components/
│               │   ├── context/
│               │   ├── pages/
│               │   ├── services/
│               │   └── styles/
│               ├── package.json
│               └── vite.config.ts

```

2.3 Configuración de la Base de Datos

```

1  -- Crear la base de datos antes de iniciar el backend
2  CREATE DATABASE nexo_db;
3  CREATE USER nexo_user WITH PASSWORD 'tu_password';
4  GRANT ALL PRIVILEGES ON DATABASE nexo_db TO nexo_user;

```

Flyway ejecuta automáticamente las migraciones al arrancar el backend. No es necesario ejecutar ningún SQL manual adicional.

2.4 Variables de Entorno

2.4.1 Backend — nexo-backend/.env

```
1  # Base de datos
2  DB_HOST=localhost
3  DB_PORT=5432
4  DB_NAME=nexo_db
5  DB_USER=postgres
6  DB_PASS=tu_password
7
8  # JWT -- cambiar en produccion por un valor seguro de minimo 32
   caracteres
9  JWT_SECRET=cambia-este-secreto-por-uno-seguro-de-produccion
10 JWT_EXPIRATION=86400000 # 24 horas en milisegundos
11
12 # Correo electronico (Gmail)
13 MAIL_USERNAME=nexoud9@gmail.com
14 MAIL_PASSWORD=contrase a_de_aplicacion_gmail
15
16 # Directorio de subida de archivos
17 UPLOAD_DIR=uploads
18
19 # API de rutas HERE (transporte publico)
20 HERE_API_KEY=tu_clave_de_here_api
21
22 # CORS -- lista de origenes permitidos separados por coma
23 CORS_ORIGINS=http://localhost:3000,http://localhost:5173
24
25 # Puerto del servidor (opcional, default 8080)
26 PORT=8080
```

2.4.2 Frontend — nexo-frontend/.env

```
1  VITE_API_URL=/api/v1
```

El frontend usa un proxy de Vite en desarrollo que redirige `/api` → `http://localhost:8080`, por lo que no es necesario apuntar directamente al backend.

2.5 Pasos para Levantar el Entorno Local

Paso 1 — Clonar el repositorio

```
1  git clone <url-del-repositorio>
2  cd NEXO-UD
```

Paso 2 — Iniciar el backend

```

1 cd nexo-backend
2
3 # Copiar y configurar las variables de entorno
4 cp .env.example .env
5 # Editar .env con tus valores locales
6
7 # Compilar y ejecutar (Flyway migra automaticamente al iniciar)
8 ./mvnw spring-boot:run
9
10 # Alternativa: compilar el JAR y ejecutarlo
11 ./mvnw clean package -DskipTests
12 java -jar target/nexo-backend-0.0.1-SNAPSHOT.jar

```

El backend queda disponible en: <http://localhost:8080>

Paso 3 — Iniciar el frontend

```

1 cd ../nexo-frontend
2 npm install
3 npm run dev

```

El frontend queda disponible en: <http://localhost:3000>

Paso 4 — Credenciales de prueba (sembradas por Flyway)

Email	Password	Rol
admin@udistrital.edu.co	admin123	ADMINISTRADOR
radicador.avisos@udistrital.edu.co	admin123	RADICADOR_AVISOS
radicador.bienestar@udistrital.edu.co	admin123	RADICADOR_BIENESTAR
radicador.sedes@udistrital.edu.co	admin123	RADICADOR_SEDES
radicador.calendario@udistrital.edu.co	admin123	RADICADOR_CALEDARIO

Cuadro 4: Credenciales de prueba sembradas por Flyway

Los estudiantes se registran con correo @udistrital.edu.co y código estudiantil de 11 dígitos.

Paso 5 — Build de producción del frontend

```

1 cd nexo-frontend
2 npm run build
3 # El output queda en nexo-frontend/build/

```


3. Topología del Dominio y Modelo de Datos

3.1 Entidades Principales

El esquema fue creado mediante Flyway (V1 a V13). A continuación se describen las entidades centrales y sus relaciones.

3.1.1 users — Usuario

Campo	Tipo	Notas
id	BIGSERIAL PK	—
email	VARCHAR(255) UNIQUE	Debe ser @udistrital.edu.co
nickname	VARCHAR(50) UNIQUE	Alias público del usuario
password_hash	VARCHAR(255)	BCrypt
student_code	VARCHAR(20) UNIQUE	11 dígitos; codifica año, semestre, carrera
entry_semester	VARCHAR(10)	Derivado del código estudiantil
active	BOOLEAN	false hasta verificar email
created_at	TIMESTAMP	—
updated_at	TIMESTAMP	—

Cuadro 5: Entidad users

3.1.2 user_roles — Roles de Usuario

Campo	Tipo	Notas
id	BIGSERIAL PK	—
user_id	FK → users	Constraint UNIQUE(user_id, role_name)
role_name	VARCHAR(50)	Ver enum RoleName
assigned_at	TIMESTAMP	—
assigned_by	VARCHAR(255)	Email del admin que asignó

Cuadro 6: Entidad user_roles

Enum RoleName:

```
ADMINISTRADOR
ESTUDIANTE
RADICADOR_AVISOS
RADICADOR_BIENESTAR
```

```
RADICADOR_SEDES
RADICADOR_CALENDARIO
```

3.1.3 verification_codes — Códigos de Verificación

Tabla compartida para verificar email, cambio de nickname y recuperación de contraseña.

Campo	Tipo	Notas
id	BIGSERIAL PK	—
email	VARCHAR(255)	Destinatario
code	VARCHAR(10)	6 dígitos
purpose	VARCHAR(50)	ACTIVATION, PASSWORD_RESET, NICKNAME_CHANGE
expires_at	TIMESTAMP	TTL configurable (default 10 min)
attempts	INTEGER	Máximo 3 intentos
consumed	BOOLEAN	Invalidado tras uso exitoso

Cuadro 7: Entidad verification_codes

3.1.4 Oferta Académica — Jerarquía

```
academic_offers (1)
  subjects (N)
    subject_groups (N) -> docente, inscritos, cupo
    time_blocks (N) -> dia, hora_inicio,
    hora_fin, salon
```

3.1.5 Horarios Guardados

```
schedules (1) -> usuario, nombre, semestre, creditos_totales,
  archivado
  schedule_blocks (N)
    Automatico: group_id FK, subject_id FK
    Manual: label, day, start_time, end_time, color
```

3.1.6 Campus

```
campuses (1) -> nombre, facultad, direccion, latitud, longitud
  classrooms (N) -> nombre, edificio, piso, is_lab
  classroom_photos (N) -> url, uploaded_by,
  uploaded_at
```

3.1.7 Tipos Enum de Dominio

CalendarEventType INSCRIPCION, INICIO_CLASES, FIN_CLASES, PARCIAL, FESTIVO, PARO, OTRO

WelfareCategory APOYO_ALIMENTARIO, BECAS, SALUD_MENTAL, SERVICIOS_SALUD, OTRO

ReportStatus PENDIENTE, EN_REVISION, RESUELTO, DESCARTADO

ReportType ERROR_HORARIO, CAMBIO_SALON, INFORMACION_INCORRECTA, OTRO

3.2 Diagrama de Relaciones Clave

```

users (1) ----- (N) user_roles
users (1) ----- (N) user_academic_progress (1) ----- (N)
    user_subject_progress
user_academic_progress (N) ----- (1) study_plans
user_subject_progress (N) ----- (1) curriculum_subjects
study_plans (1) ----- (N) curriculum_subjects
academic_offers (1) ----- (N) subjects (1) ----- (N)
    subject_groups (1) ----- (N) time_blocks
users (1) ----- (N) schedules (1) ----- (N) schedule_blocks
campuses (1) ----- (N) classrooms (1) ----- (N) classroom_photos
  
```

4. Diseño de la Capa de Servicios y Lógica de Negocio

4.1 Inventario de Servicios

Interfaz / Implementación	Módulo	Responsabilidad
AuthService / AuthServiceImpl	auth	Registro, login, verificación, reset de contraseña
UserService / UserServiceImpl	user	Perfil, búsqueda, nickname, eliminación, activación
RoleService / RoleServiceImpl	user	Asignación y revocación de roles
StudyProgressService	user	Inscripción a carreras, seguimiento de materias
AcademicOfferService	academic	Carga desde Excel, activación de oferta
StudyPlanService	academic	Listado y consulta de planes de estudio
CurriculumSubjectService	academic	CRUD de materias del pensum
ScheduleService	schedule	CRUD de horarios, detección de conflictos
ScheduleExportService	schedule	Exportación a PDF e imagen PNG
AnnouncementService	content	CRUD de avisos universitarios
CalendarEventService	content	CRUD de eventos del calendario
WelfareService	content	CRUD de recursos de bienestar
ReportService	report	Creación y gestión de reportes
CampusService	campus	CRUD de sedes, aulas, fotos
EmailService	shared	Envío de correos vía SMTP Gmail
JwtService	shared	Generación y validación de JWT

Cuadro 8: Inventario de servicios del backend

4.2 Lógica de Negocio Destacada

4.2.1 Registro de Estudiante (AuthServiceImpl.register)

El código estudiantil de 11 dígitos contiene información codificada:

```

Digitos 0-3  -> Año de ingreso      (ej: "2023")
Digito 4     -> Semestre (1=primero, 3=segundo)
Digitos 5-7  -> Código de carrera   (ej: "005" =
    Ingeniería Civil)
Digitos 8-10 -> Número de lista

```

Al registrarse, el sistema:

1. Valida formato del código (11 dígitos numéricos exactos).

2. Deriva `entry_semester` del código (ej: "2023-1").
3. Busca el `StudyPlan` por los dígitos 5-7.
4. Crea el usuario inactivo y envía un código de verificación al correo.

4.2.2 Sistema de Códigos de Verificación

```
TTL por defecto: 10 minutos (nexo.code.ttl-minutes)
Maximo intentos: 3 (nexo.code.max-attempts)
Propósitos: ACTIVATION | PASSWORD_RESET | NICKNAME_CHANGE
```

- Un solo código activo por email + propósito a la vez.
- Se invalida (`consumed = true`) al usarse correctamente.
- Se reporta un error genérico si se superan los intentos (no se revela cuántos quedan).

4.2.3 Exportación de Horario (`ScheduleExportServiceImpl`)

- **PDF:** Usa la librería OpenPDF (fork de iText) para generar una tabla visual del horario.
- **Imagen PNG:** Renderiza el horario como imagen en memoria.

4.3 Gestión de Transacciones

- Todas las operaciones de escritura usan `@Transactional`.
- Las consultas de solo lectura usan `@Transactional(readOnly = true)`.
- Los controllers que hacen operaciones directas sobre repositorios también usan `@Transactional`.

4.4 Excepciones Personalizadas

`NexoException` es la excepción de negocio central. Se lanza desde los servicios y es capturada por `GlobalExceptionHandler`.

```
1 // Fabrica estatica -- no se instancia directamente
2 NexoException.notFound("Mensaje")      -> HTTP 404
3 NexoException.badRequest("Mensaje")    -> HTTP 400
4 NexoException.conflict("Mensaje")      -> HTTP 409
5 NexoException.forbidden("Mensaje")     -> HTTP 403
```

Estructura del error de respuesta:

```
1 {  
2   "status": 400,  
3   "message": "Validacion fallida",  
4   "details": "nickname: El nickname debe tener entre 3 y 30  
               caracteres",  
5   "timestamp": "2025-05-13T10:30:00"  
6 }
```

5. Contratos de Integración (API Layer)

5.1 Autenticación JWT

Todos los endpoints protegidos requieren el header:

```
Authorization: Bearer <jwt_token>
```

El token se obtiene al hacer login exitoso (POST /api/v1/auth/login).

5.2 Endpoints Públicos (sin autenticación)

POST /api/v1/auth/**	-> Registro, login , recuperacion de contraseña
GET /api/v1/study-plans	-> Lista de planes de estudio
GET /api/v1/semesters/active	-> Semestre activo
GET /api/v1/schedules/offer/**	-> Materias de la oferta activa
POST /api/v1/schedules/validate-conflicts	
GET /api/v1/announcements/**	-> Avisos
GET /api/v1/calendar/**	-> Calendario
GET /api/v1/welfare/**	-> Bienestar
GET /api/v1/campus/**	-> Sedes y aulas
POST /api/v1/campus/route	-> Calculo de ruta (HERE Transit)
GET /actuator/**	-> Metricas de Spring Actuator

5.3 Resumen de Controllers y Rutas

5.3.1 AuthController — /api/v1/auth

Método	Endpoint	Descripción	Auth
POST	/register	Registrar nuevo estudiante	Público
POST	/verify-code	Activar cuenta con código	Público
POST	/resend-code	Reenviar código de verificación	Público
POST	/login	Iniciar sesión → retorna JWT	Público
POST	/logout	Cerrar sesión (stateless)	Público
POST	/forgot-password	Solicitar código de reset	Público
POST	/reset-password	Cambiar contraseña con código	Público

Cuadro 9: Endpoints de AuthController

5.3.2 UserController — /api/v1/users

Método	Endpoint	Descripción	Rol
GET	/me	Perfil del usuario autenticado	Autenticado
POST	/me/nickname/request-code	Solicitar código para cambiar nickname	Autenticado
PATCH	/me/nickname	Cambiar nickname con código	Autenticado
DELETE	/me	Eliminar cuenta propia	ESTUDIANTE
GET	/ {id}	Perfil de usuario por ID	ADMINISTRADOR
GET	/search?email=	Buscar usuario por email	ADMINISTRADOR

Cuadro 10: Endpoints de UserController

5.3.3 SchedulePlannerController — /api/v1/schedules

Método	Endpoint	Descripción	Rol
GET	/offer/subjects	Materias de la oferta activa	Público
POST	/validate-conflicts	Validar conflictos de horario	Público
GET	/	Mis horarios guardados	ESTUDIANTE
POST	/	Crear horario	ESTUDIANTE
GET	/scheduleId	Detalle de horario	ESTUDIANTE
PUT	/scheduleId	Actualizar horario	ESTUDIANTE
DELETE	/scheduleId	Eliminar horario	ESTUDIANTE
PATCH	/scheduleId/archive	Archivar/desarchivar	ESTUDIANTE

Cuadro 11: Endpoints de SchedulePlannerController

5.3.4 CampusController — /api/v1/campus

Método	Endpoint	Descripción	Rol
GET	/	Listar sedes	Público
GET	/campusId	Detalle de sede	Público
POST	/	Crear sede	RAD_SEDES / ADMIN
PUT	/campusId	Actualizar sede	RAD_SEDES / ADMIN
DELETE	/campusId	Eliminar sede	RAD_SEDES / ADMIN
GET	/campusId/classrooms	Listar aulas	Público
POST	/campusId/classrooms	Crear aula	RAD_SEDES / ADMIN
POST	/route	Calcular ruta (HERE Transit)	Público

Cuadro 12: Endpoints de CampusController (resumen)

5.3.5 ReportController — /api/v1/reports

Método	Endpoint	Descripción	Rol
POST	/	Crear reporte	ESTUDIANTE
GET	/my	Mis reportes	ESTUDIANTE
GET	/	Todos los reportes	ADMINISTRADOR
GET	/ {id}	Detalle de reporte	ADMINISTRADOR
PATCH	/ {id}/status	Cambiar estado de reporte	ADMINISTRADOR

Cuadro 13: Endpoints de ReportController

5.3.6 SemesterController

Método	Endpoint	Descripción	Rol
GET	/api/v1/semesters/active	Semestre activo	Público
GET	/api/v1/admin/semesters	Listar semestres	ADMIN
POST	/api/v1/admin/semesters	Crear semestre	ADMIN
PATCH	/api/v1/admin/semesters/ {id}/activate	Activar semestre	ADMIN
DELETE	/api/v1/admin/semesters/ {id}	Eliminar semestre	ADMIN

Cuadro 14: Endpoints de SemesterController

5.4 Paginación

```
GET
/api/v1/admin/users?email=garcia&page=0&size=20&sort=createdAt,desc
```

```
1 {
2   "content": [...],
3   "totalElements": 150,
4   "totalPages": 8,
5   "number": 0,
6   "size": 20
7 }
```

5.5 DTOs — Ejemplos de Request/Response

5.5.1 Login

```
1 // POST /api/v1/auth/login
2 // Request
3 {
4   "email": "estudiante@udistrital.edu.co",
5   "password": "MiContrasena1!"
```

```
6 }
7
8 // Response 200
9 {
10   "token": "eyJhbGciOiJIUzI1NiJ9...",
11   "email": "estudiante@udistrital.edu.co",
12   "nickname": "estu2023",
13   "roles": ["ESTUDIANTE"]
14 }
```

5.5.2 Registro

```
1 // POST /api/v1/auth/register
2 {
3   "email": "nuevo@udistrital.edu.co",
4   "nickname": "nuevo2023",
5   "password": "ContrasenaSegura1!",
6   "studentCode": "20231005001"
7 }
8 // Response 202 (sin body -- se envia codigo al correo)
```

5.5.3 Validar conflictos de horario

```
1 // POST /api/v1/schedules/validate-conflicts
2 {
3   "groupIds": [1, 2, 3, 7]
4 }
5
6 // Response 200
7 {
8   "hasConflicts": true,
9   "conflicts": [
10     {
11       "group1Id": 1,
12       "group2Id": 3,
13       "day": "LUNES",
14       "overlap": "7:00 - 9:00"
15     }
16   ]
17 }
```

5.6 Capa de API en el Frontend

`nexo-frontend/src/services/api.ts` organiza todas las llamadas al backend en 13 módulos exportados:

```
1 export const authApi      = { register, login, logout,
   verifyCode, resendCode,
2                               forgotPassword, resetPassword }
3 export const userApi      = { getMyProfile }
```

```

4 export const scheduleApi      = { getOfferSubjects, listMySchedules,
   createSchedule,
5                                   getSchedule, updateSchedule,
                                   deleteSchedule,
6                                   archiveSchedule,
                                   validateConflicts, exportPdf,
                                   exportImage }
7 export const announcementsApi = { listAll, getById, create,
   update, delete }
8 export const welfareApi       = { listAll, getById, create, update,
   delete }
9 export const calendarApi      = { listAll, getById, create, update,
   delete }
10 export const campusApi        = { listAll, getById, create, update,
   delete,
11                                   listClassrooms, addClassroom,
                                   updateClassroom,
12                                   deleteClassroom, addPhoto,
                                   deletePhoto }
13 export const adminApi         = { listUsers, getUserById,
   setUserStatus,
14                                   listRoles, assignRole, revokeRole,
15                                   listStudyPlans, listCurriculum,
                                   addSubject,
16                                   updateSubject, deleteSubject }
17 export const reportApi        = { create, listMine, listAll,
   getById, updateStatus }
18 export const academicOfferApi = { listAll, getActive, upload,
   activate, delete }
19 export const semesterApi      = { getActive, listAll, create,
   activate, delete }
20 export const studyPlanApi     = { listAll }
21 export const routeApi         = { getRoute }

```

6. Estrategia de Pruebas

6.1 Stack de Pruebas

Herramienta	Versión	Propósito
JUnit 5 (Jupiter)	incluido en Boot 3.5	Framework de pruebas
Mockito	incluido en Boot 3.5	Mocking de dependencias
Spring Security Test	incluido	Simular usuarios/roles en tests web
AssertJ	incluido en Boot 3.5	Aserciones fluidas
MockMvc	incluido	Tests de capa web sin servidor real

Cuadro 15: Stack de pruebas

6.2 Convenciones de Nomenclatura de Tests

```

1  @Nested
2  @DisplayName("nombreDelMetodo()")
3  class NombreDelMetodo {
4
5      @Test
6      @DisplayName("Debe <resultado esperado> cuando <condicion>")
7      void debe_resultado_cuando_condicion() {
8          // Arrange
9          when(mockDependencia.metodo(any())).thenReturn(valorMock);
10
11         // Act
12         ResultType resultado = servicio.metodo(parametro);
13
14         // Assert
15         assertThat(resultado).isNotNull();
16         verify(mockDependencia, times(1)).metodo(any());
17     }
18 }

```

6.3 Comandos para Ejecutar Tests

```

1  # Ejecutar todos los tests
2  cd nexos-backend
3  ./mvnw test
4
5  # Ejecutar una clase especifica
6  ./mvnw test -Dtest=AuthServiceImplTest
7
8  # Ejecutar un metodo especifico
9  ./mvnw test
10     -Dtest=AuthServiceImplTest#debeRetornarJwt_cuandoCredencialesSonCorrectas
11
12 # Saltar tests al compilar
13 ./mvnw clean package -DskipTests

```

6.4 Tests con Spring Security (@WebMvcTest)

Dos errores comunes al escribir @WebMvcTest con @PreAuthorize:

- **Error 1:** Excluir SecurityAutoConfiguration impide que @PreAuthorize evalúe roles → los tests de 403 siempre pasan.
- **Error 2:** No declarar @MockBean JwtAuthFilter hace que Spring intente inicializar el filtro y falle el contexto.

Solución correcta:

```

1  @WebMvcTest(UserController.class)
2  class UserControllerTest {

```

```

3
4     @MockBean private JwtAuthFilter jwtAuthFilter;
5     @MockBean private JwtService jwtService;
6     @MockBean private UserService userService;
7
8     @Test
9     @WithMockUser(roles = "ADMINISTRADOR")
10    void getById_rolAdmin_retorna200() { ... }
11
12    @Test
13    @WithMockUser(roles = "ESTUDIANTE")
14    void getById_sinPermiso_retorna403() { ... }
15 }

```

7. Estándares y Flujo de Contribución

7.1 Convenciones de Código

7.1.1 Backend (Java)

Elemento	Convención	Ejemplo
Clases	PascalCase	UserServiceImpl
Métodos	camelCase	getMyProfile
Variables	camelCase	userId, emailRequest
Constantes	UPPER_SNAKE_CASE	MAX_ATTEMPTS
Paquetes	lowercase, singular	com.kumorai.nexo.user.service
Enums	PascalCase + UPPER_CASE	RoleName.ADMINISTRADOR
Entidades JPA	PascalCase, singular	UserAcademicProgress
Tablas BD	snake_case, plural	user_academic_progress
Mensajes de error	En español, sin detalles internos	Usuario no encontrado"

Cuadro 16: Convenciones de código en Java

7.1.2 Estructura de Cada Módulo

```

modulo/
    controller/    -> Solo mapeo HTTP; delega toda logica al
    service        -> Interfaz + Impl; toda la logica de
    service/       -> Interfaz + Impl; toda la logica de
negocio aqui
    entity/        -> Clases @Entity + enums del dominio
    repository/    -> Interfaces que extienden JpaRepository
    dto/           -> Records o clases de request/response;
sin logica

```

7.2 Estructura de Ramas

```
main          -> Rama de produccion. Solo se toca con PRs
  aprobados.
develop       -> Integracion continua. Base de todos los feature
  branches.
feature/xxx   -> Nueva funcionalidad (ej:
  feature/exportar-horario-pdf)
fix/xxx       -> Correccion de bug (ej:
  fix/error-validacion-codigo)
chore/xxx     -> Mantenimiento (ej:
  chore/actualizar-dependencias)
docs/xxx      -> Solo documentacion (ej: docs/agregar-manual-dev)
```

7.3 Formato de Commits (Conventional Commits)

```
<tipo>(<alcance>): <descripcion en imperativo, minusculas>

feat(schedule): agregar exportacion de horario a PNG
fix(auth): corregir validacion de codigo expirado
chore(deps): actualizar Spring Boot a 3.5.14
docs(api): documentar endpoints de bienestar
test(user): agregar tests de controller para UserController
refactor(campus): extraer logica de ruta a servicio dedicado
```

7.4 Checklist Antes de Hacer PR

El código compila sin errores: `./mvnw clean compile`

Todos los tests existentes pasan: `./mvnw test`

Se agregaron tests para la nueva funcionalidad

No hay credenciales, secrets ni `.env` en el commit

Los endpoints nuevos están documentados en este manual

Las migraciones Flyway siguen la numeración correlativa (`V14__descripcion.sql`)

El frontend compila sin errores de TypeScript: `npm run build`

La PR apunta a `develop`, no directamente a `main`

7.5 Agregar una Nueva Migración Flyway

```
1 # Convencion de nombre:
2 # V<numero>__<descripcion_en_snake_case>.sql
3 touch
  nex-backend/src/main/resources/db/migration/V14__add_notification_table.sql
```

Nunca modificar una migración ya aplicada (V1 a V13). Flyway detecta cambios en migraciones existentes y rechaza el arranque. Siempre crear una nueva versión.

7.6 Agregar un Nuevo Módulo de Dominio

1. Crear el paquete `com.kumorai.nexo.<módulo>/` con subcarpetas `controller`, `service`, `entity`, `repository`, `dto`.
2. Crear la entidad `@Entity` y la migración Flyway correspondiente.
3. Crear la interfaz de servicio e implementación.
4. Crear el controller `@RestController` con el prefijo `/api/v1/<módulo>`.
5. Registrar los endpoints públicos en `SecurityConfig.java` si aplica.
6. Crear los tests unitarios del servicio en `src/test/.../service/`.
7. Crear los tests del controller en `src/test/.../controller/`.
8. Agregar el módulo de API al frontend en `src/services/api.ts`.

8. Gestión de Estado del Frontend

8.1 Estrategia General

El frontend **no usa ninguna librería externa de estado global**. La estrategia se apoya en tres pilares:

Estado Global — React Context API (2 contextos)

`AuthContext` → usuario, token, sesión. `ThemeContext` → tema dark/light.

Estado Local — useState por componente/página

loading, error, datos de formulario, filtros.

Persistencia — localStorage (3 claves)

`nexoud_token` (JWT), `nexoud_user` (perfil en caché), `nexoud_local_data` (datos locales por email).

8.2 AuthContext — Flujo de Restauración de Sesión

Al recargar la página, `AuthContext` ejecuta la siguiente secuencia:

1. Lee `nexoud_token` de `localStorage`.
2. Si existe, llama `GET /api/v1/users/me` para obtener el perfil.
3. Actualiza `nexoud_user` en `localStorage`.
4. Establece `isRestoring = false` y renderiza la app.

5. Si no existe token, establece `user = null`.

Clave	Contenido	Cuándo se actualiza
<code>nexoud_token</code>	JWT string	En login; se borra en logout
<code>nexoud_user</code>	JSON del perfil completo	En login y en <code>updateUser()</code>
<code>nexoud_local_data</code>	<code>{[email]: {materiasVistas, horariosGuardados}}</code>	En actualizaciones locales

Cuadro 17: Claves de localStorage

8.3 Limitaciones Conocidas y Mejoras Futuras

Limitación actual	Impacto	Mejora recomendada
Sin caché de API	Requests repetidas al navegar	Adoptar React Query / SWR
Sin refresh token	Logout forzado cada 24 h	Implementar refresh token endpoint
25+ useState por página compleja	Difícil de testear y mantener	Extraer lógica a custom hooks
Sin error boundary global	Errores silenciosos en producción	<code>ErrorBoundary</code> + contexto de error
Filtros en state local	URL no comparte estado de filtros	Migrar filtros a URL params

Cuadro 18: Deuda técnica y mejoras sugeridas

9. Infraestructura y Despliegue

9.1 Estado Actual de la Infraestructura

Componente	Plataforma	Estado
Backend container	Docker (multi-stage, JDK 21)	Configurado
Backend hosting	Render.com (free tier)	Desplegado
Frontend hosting	Vercel	Desplegado
Base de datos	PostgreSQL (Render managed)	Configurado
CI/CD	No configurado	Pendiente
Monitoreo	No configurado	Pendiente
Logs centralizados	Spring Boot default	Pendiente

Cuadro 19: Estado de infraestructura

9.2 Dockerfile del Backend

```

1  # Stage 1: Compilacion
2  FROM eclipse-temurin:21-jdk-jammy AS build
3  WORKDIR /app
4  COPY .mvn/ .mvn/
5  COPY mvnw ./
6  COPY pom.xml ./
7  RUN chmod +x mvnw
8  RUN ./mvnw dependency:go-offline
9  COPY src ./src
10 RUN ./mvnw clean install -DskipTests
11
12 # Stage 2: Imagen de produccion (solo JRE, sin Maven ni JDK)
13 FROM eclipse-temurin:21-jre-jammy
14 WORKDIR /app
15 COPY --from=build /app/target/*.jar app.jar
16 # -Xmx256m: limite de heap para free tier de Render
17 ENTRYPOINT ["java", "-Xmx256m", "-jar", "app.jar"]

```

El Dockerfile usa JDK/JRE 21, pero el `pom.xml` declara `<java.version>17</java.version>`. El código compila con `source/target 17`; el JRE 21 es retrocompatible.

9.3 Variables de Entorno Requeridas en Producción

```

1  # Base de datos
2  DB_HOST=<host-postgres>
3  DB_PORT=5432
4  DB_NAME=nexo_db
5  DB_USERNAME=<usuario>
6  DB_PASSWORD=<password>
7
8  # JWT

```

```
9 JWT_SECRET=<minimo 32 caracteres aleatorios>
10 JWT_EXPIRATION_MS=86400000
11
12 # Email (Gmail con App Password)
13 MAIL_HOST=smtp.gmail.com
14 MAIL_PORT=587
15 MAIL_USERNAME=nexoud9@gmail.com
16 MAIL_PASSWORD=<contrasena de aplicacion>
17
18 # Servidor
19 PORT=8080
20
21 # Integraciones externas
22 HERE_API_KEY=<clave de HERE Developer>
```

9.4 CI/CD — Estado y Recomendación

Actualmente no existe ningún pipeline de CI/CD. Estructura recomendada con GitHub Actions:

```
.github/workflows/
  backend-ci.yml    -> compile + test en cada PR a
    develop/main
  frontend-ci.yml   -> lint + build en cada PR
  deploy.yml        -> trigger manual o en merge a main
```

```
1 name: Backend CI
2 on:
3   push:
4     branches: [main, develop]
5     paths: ['nexo-backend/**']
6   pull_request:
7     paths: ['nexo-backend/**']
8
9 jobs:
10   test:
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v4
14       - uses: actions/setup-java@v4
15         with:
16           java-version: '17'
17           distribution: 'temurin'
18       - name: Run tests
19         working-directory: nexo-backend
20         run: ./mvnw test
```

10. Observabilidad y Logs

10.1 Configurar Niveles de Log

```

1 # Subir nivel de detalle para el paquete propio en desarrollo
2 logging.level.com.kumorai.nexo=DEBUG
3
4 # Silenciar librerías ruidosas
5 logging.level.org.hibernate.SQL=WARN
6 logging.level.org.springframework.security=INFO
7
8 # Ver queries SQL con parametros (solo desarrollo)
9 logging.level.org.hibernate.orm.jdbc.bind=TRACE

```

10.2 Endpoints de Actuator

```

# Estado general (usado como health check en Render)
GET /actuator/health
# Respuesta: { "status": "UP" }

GET /actuator/info
GET /actuator/metrics

```

10.3 Monitoreo Futuro Recomendado

Herramienta	Tier gratuito	Integración
UptimeRobot	Sí (50 monitores)	Solo necesita la URL del health check
Sentry	Sí (5k errores/mes)	<code>sentry-spring-boot-starter</code> + DSN
Grafana Cloud	Sí (limitado)	Micrometer + Prometheus
Logtail / BetterStack	Sí (1 GB/mes)	Apuntar stdout de Render a Logtail

Cuadro 20: Opciones de monitoreo recomendadas

11. Troubleshooting — Errores Comunes

11.1 Backend

Error: Connection refused a PostgreSQL

```

PSQLException: Connection refused. Check that the hostname and port are
correct...

```

Causa: PostgreSQL no está corriendo, o las credenciales en `.env` son incorrectas.

```
1 pg_isready -h localhost -p 5432
2 psql -U postgres -c "\l" | grep nexo_db
3 psql -U postgres -c "CREATE DATABASE nexo_db;"
```

Error: Flyway detecta migraciones modificadas

FlywayValidateException: Validate failed: Migration checksum mismatch for migration version 3

Causa: Se modificó un archivo `V*.sql` ya aplicado.

```
1 # NUNCA modificar migraciones aplicadas.
2 # Si la BD es desechable en desarrollo:
3 psql -U postgres -c "DROP DATABASE nexo_db; CREATE DATABASE
   nexo_db;"
```

Error: Puerto 8080 ocupado

```
1 # macOS/Linux
2 lsof -ti:8080 | xargs kill -9
3
4 # Windows
5 netstat -ano | findstr :8080
6 taskkill /PID <pid> /F
```

Error: JWT_SECRET muy corto

WeakKeyException: The signing key's size is 40 bits which is not secure enough for HS256.

```
1 # Generar un secreto seguro de 64 caracteres
2 openssl rand -hex 32
```

Error: Flyway en tests @SpringBootTest

Crear `src/test/resources/application-test.properties`:

```
1 spring.flyway.enabled=false
2 spring.datasource.url=jdbc:h2:mem:testdb
3 spring.datasource.driver-class-name=org.h2.Driver
4 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

11.2 Frontend

Error: Puerto 3000 ocupado

```
1 lsof -ti:3000 | xargs kill -9
```

Error: Llamadas a la API retornan 404 o CORS

```
1 # Verificar que el backend esta levantado
2 curl http://localhost:8080/actuator/health
3
4 # Verificar que VITE_API_URL=/api/v1 (sin host) para que el proxy
   funcione
```

Error: Dependencias de Radix UI faltantes

```
1 rm -rf node_modules package-lock.json
2 npm install
```

Error: Caché de Vite corrupta

```
1 rm -rf node_modules/.vite
2 npm run dev
```

Error: Pantalla en blanco sin errores en consola

```
1 # En DevTools > Application > Local Storage:
2 # Borrar nexoud_token y nexoud_user y recargar
3 localStorage.clear()
```